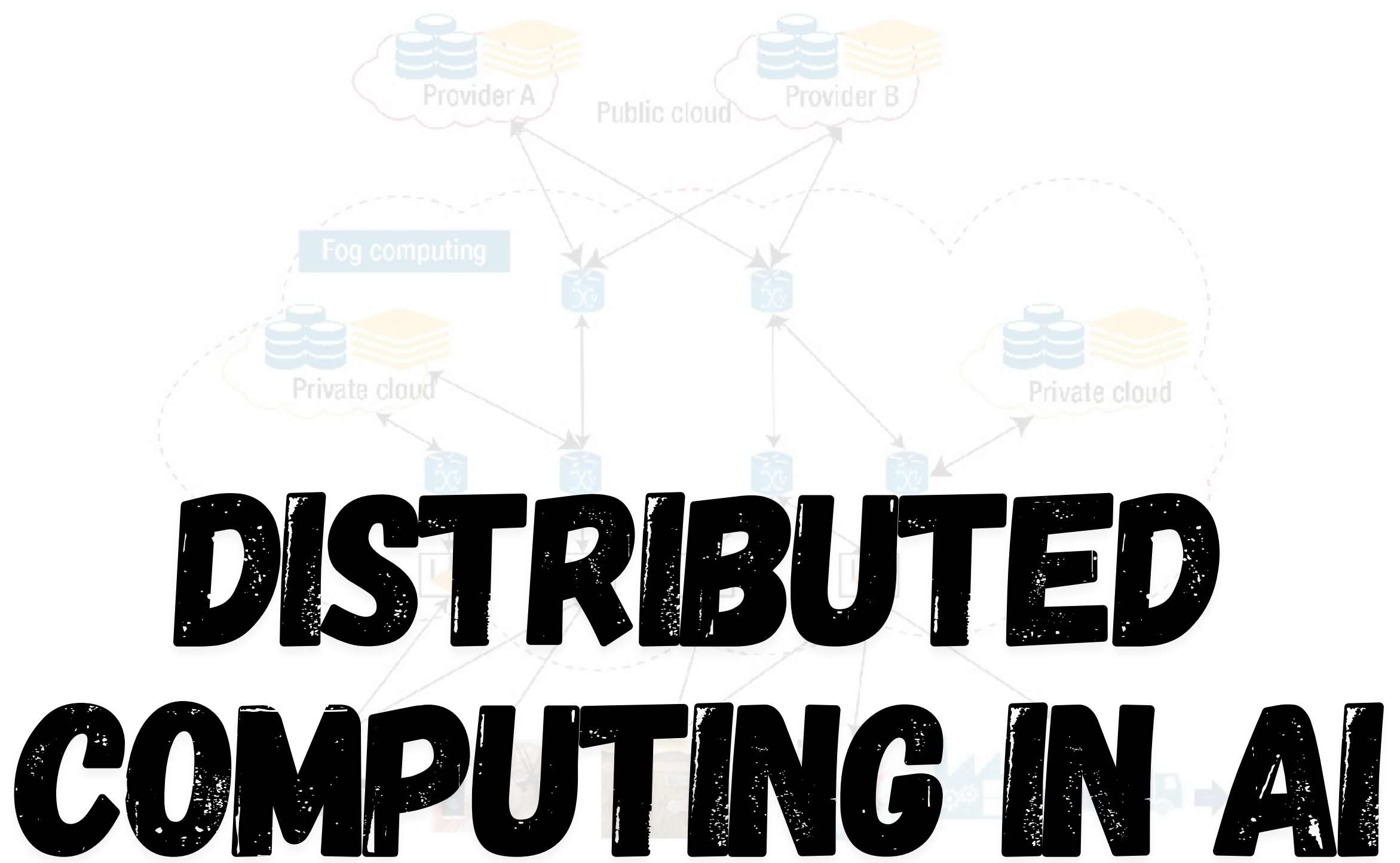




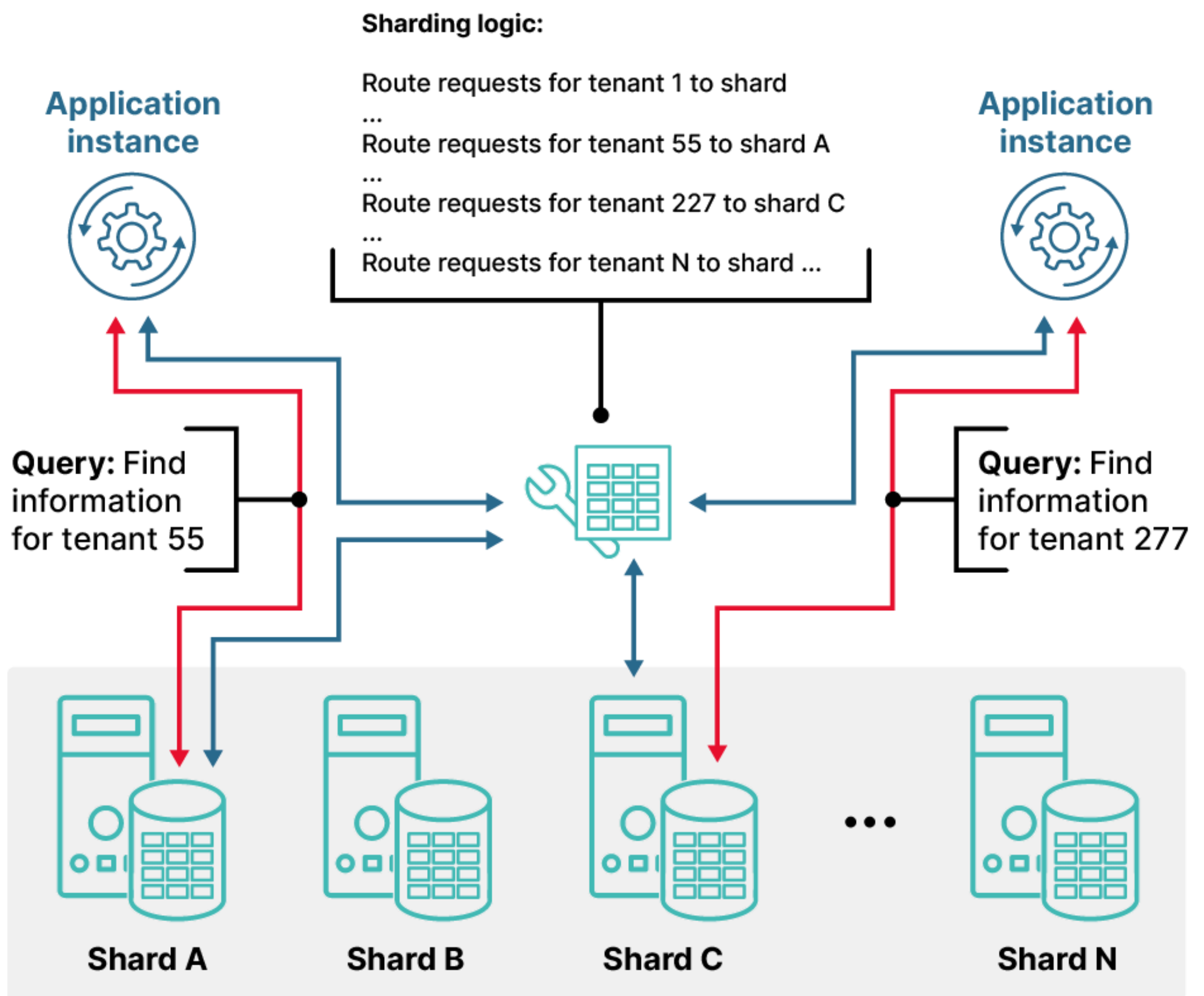
DISTRIBUTED COMPUTING IN AI

HOW WE ACTUALLY MAKE “MASSIVE MODELS” WORK

Why Distributed Computing?

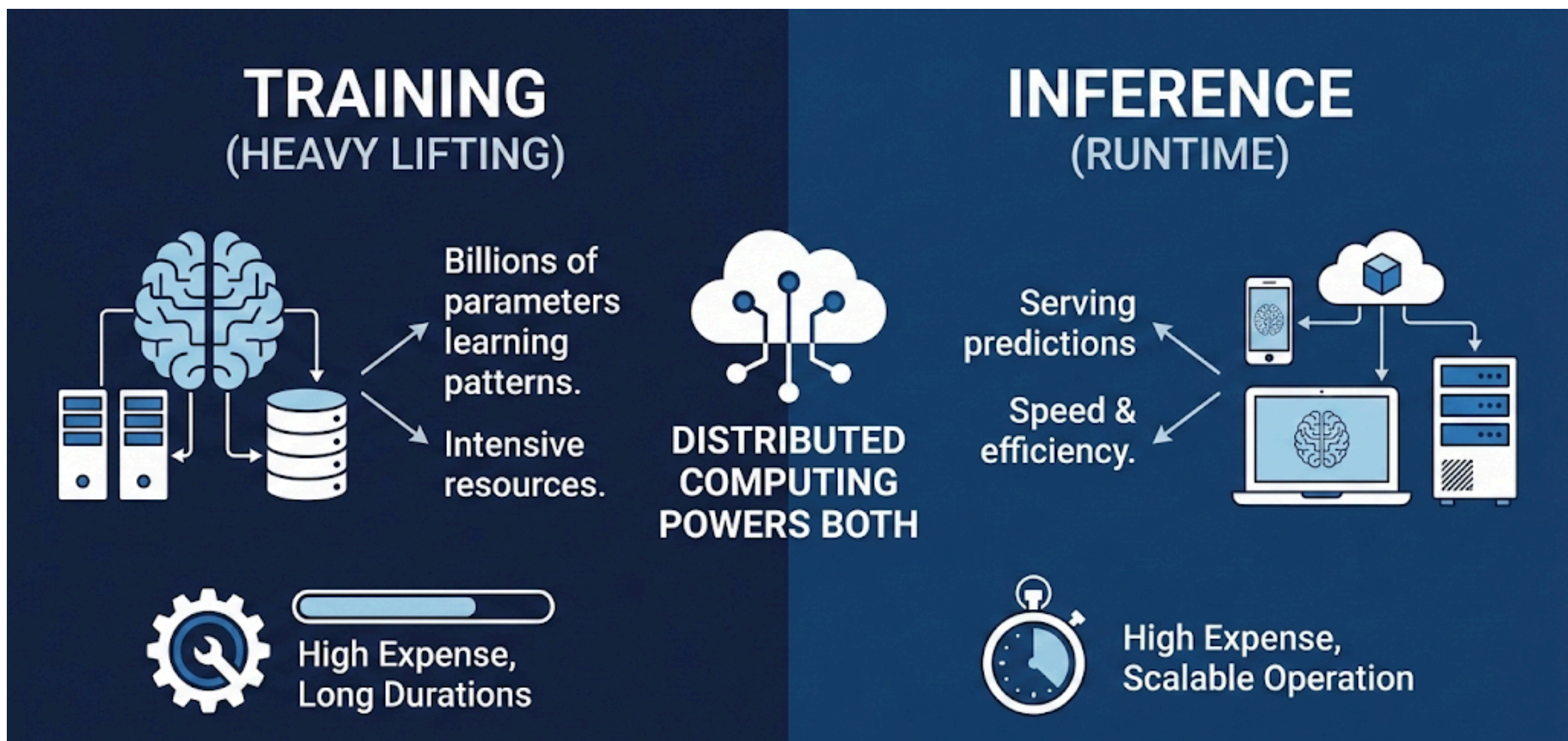
AI workloads today are too big for a single machine. Models are huge, data is insane and deadlines are real.

Distributed computing spreads work across multiple systems so AI can actually be fast, scalable, and practical.



Source

Training vs Inference



Training = Teaching the model

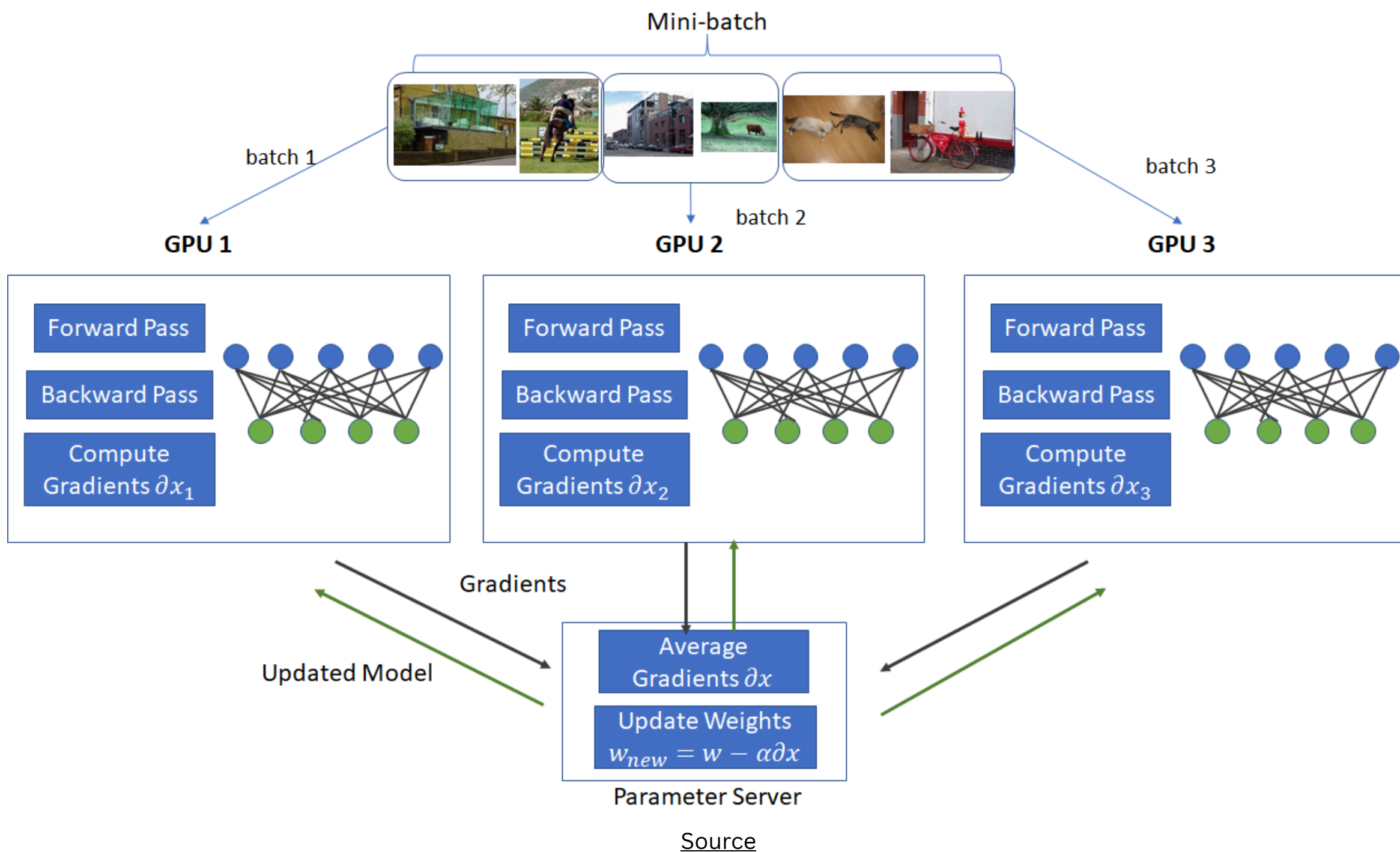
- This is where the model learns from massive datasets, adjusts billions of parameters, and repeatedly runs forward + backward passes.
- It's slow, insanely compute-heavy, needs distributed GPUs, and takes days or weeks. Think: "marathon with weights on."

Inference = Using the model

- This is when the trained model actually answers queries, generates text, detects images, or makes predictions.
- It's much lighter than training, but at scale it still needs distributed systems to handle millions of real-time requests without lag.

Data Parallelism

When the dataset is massive, we don't force one GPU to suffer. Instead, we copy the same model to many GPUs and split the data among them so everyone trains in parallel.



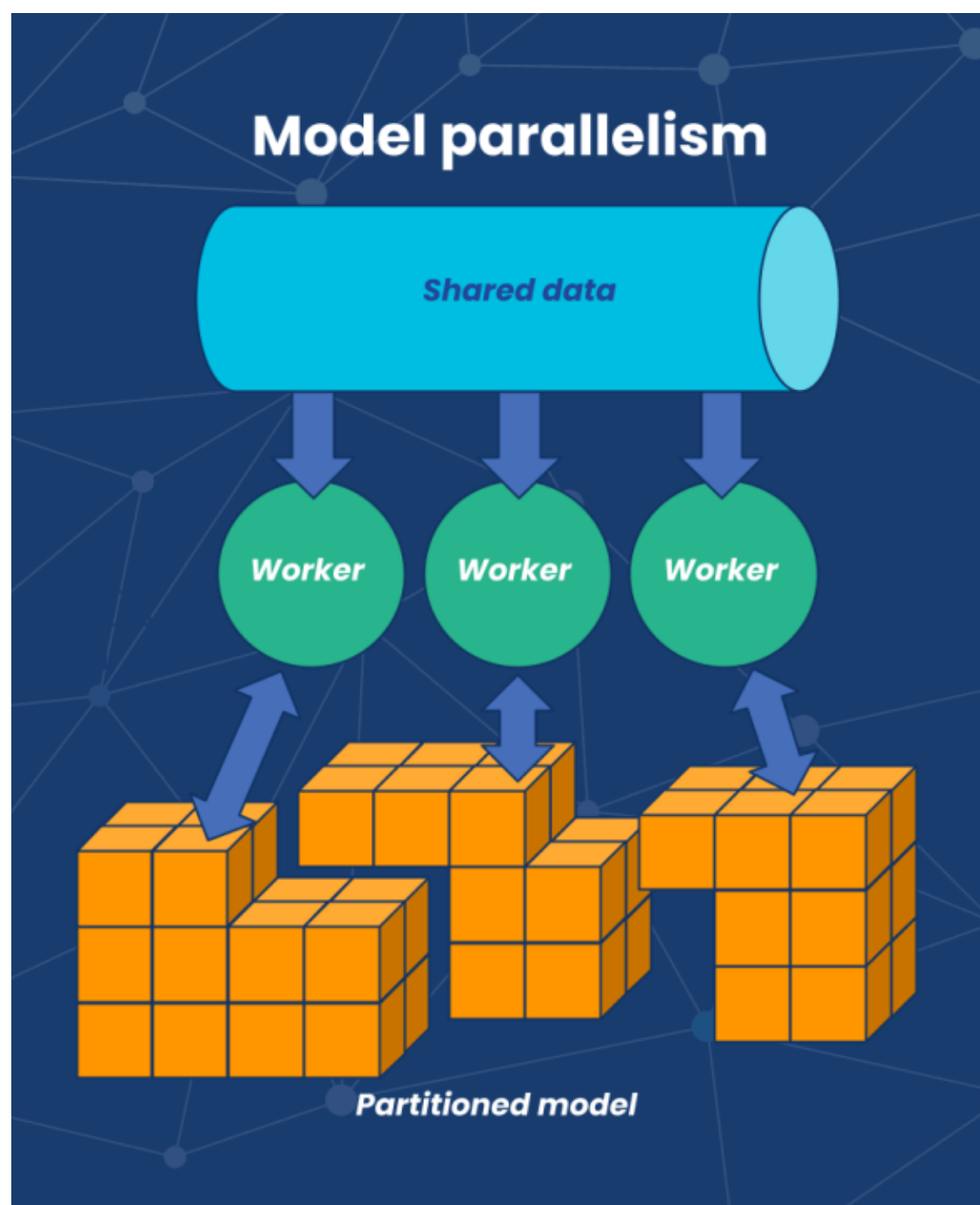
Quick takeaway:

Data Parallelism = same brain, different experiences, shared learning

What if even the model is too big for one GPU? That's where Model Parallelism steps in.

Model Parallelism

Sometimes the problem isn't the dataset, it's the model itself. Modern AI models can be huge so huge that a single GPU simply can't hold all the parameters. So instead of duplicating the model like in data parallelism, we split one big model across multiple GPUs.



Source

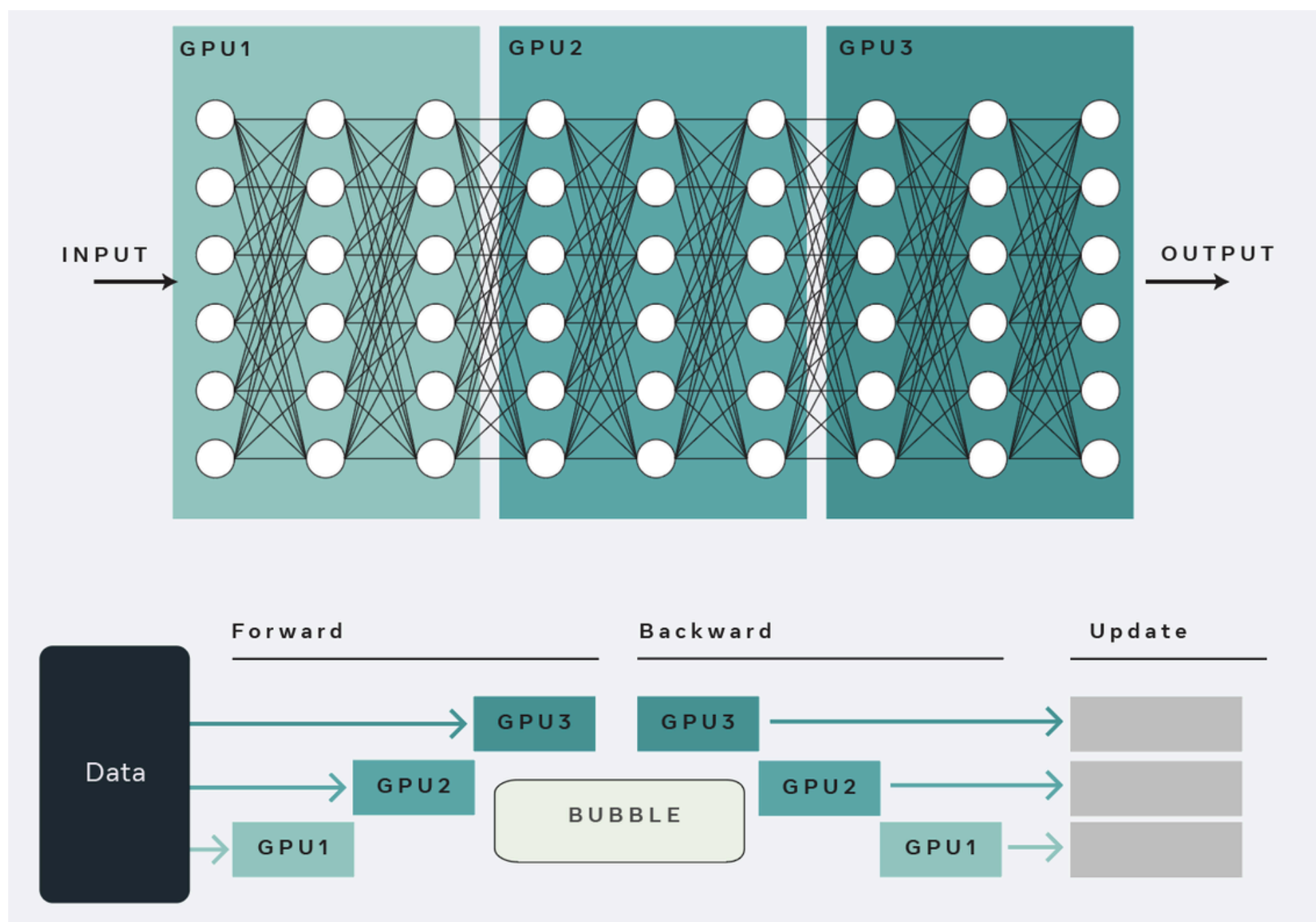
Quick takeaway:

Model Parallelism = one giant model sliced across multiple GPUs so impossible models actually become trainable.

But even splitting models isn't always enough which is why Pipeline Parallelism joins the party next.

Pipeline Parallelism

If model parallelism splits a model across GPUs, pipeline parallelism makes those GPUs work like an assembly line instead of one-at-a-time workers. Instead of waiting for one GPU to finish the entire forward pass before the next one starts, we keep all GPUs busy at once.



Source

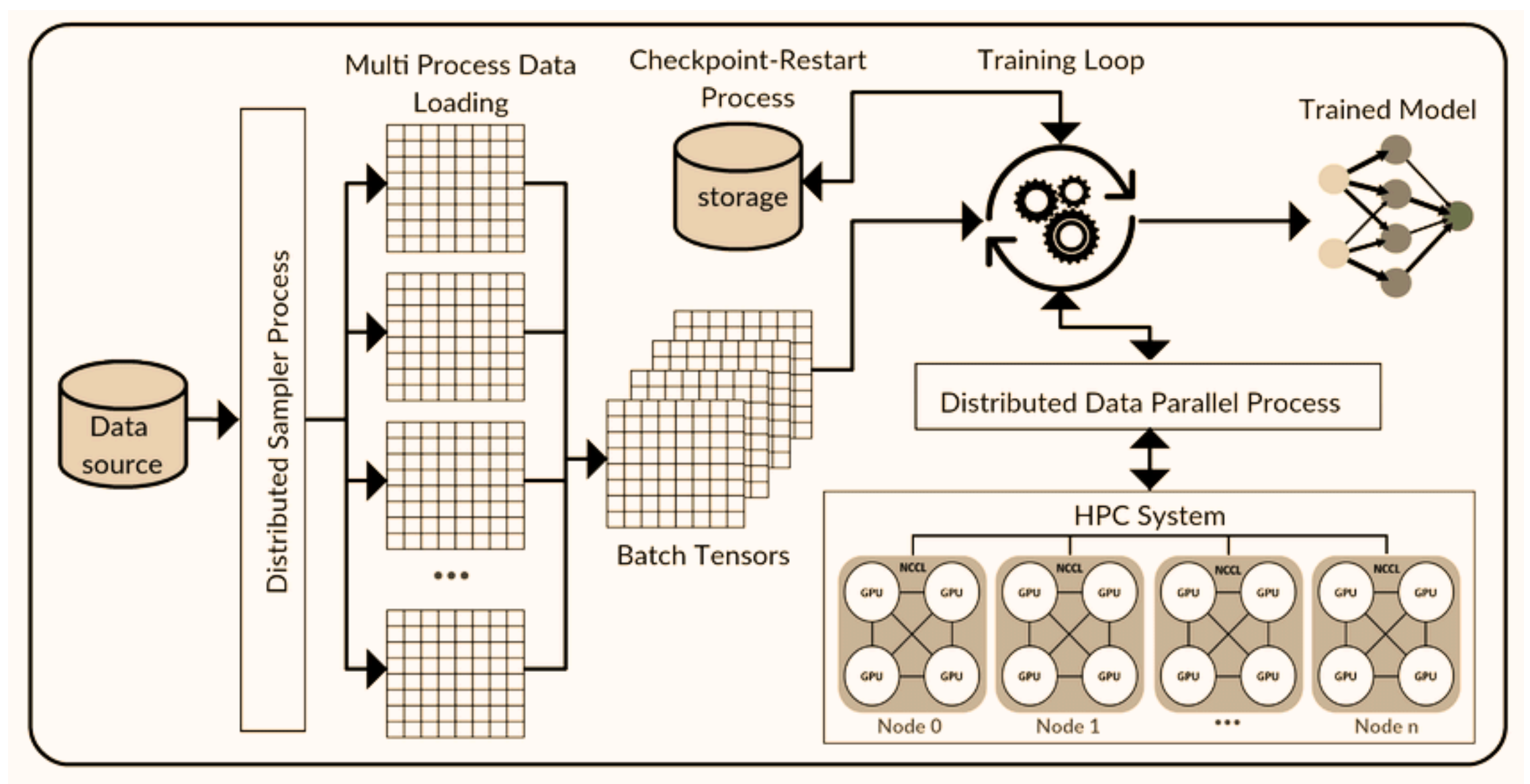
Simple takeaway:

Pipeline Parallelism = AI training as an assembly line → every GPU stays busy, everything moves faster.

Now imagine combining data parallelism + model parallelism + pipeline parallelism that's where modern AI really hits beast mode.

Distributed Training = All Three Together

Modern AI systems don't rely on just one parallelism technique. They mix Data Parallelism + Model Parallelism + Pipeline Parallelism like a well-balanced recipe.



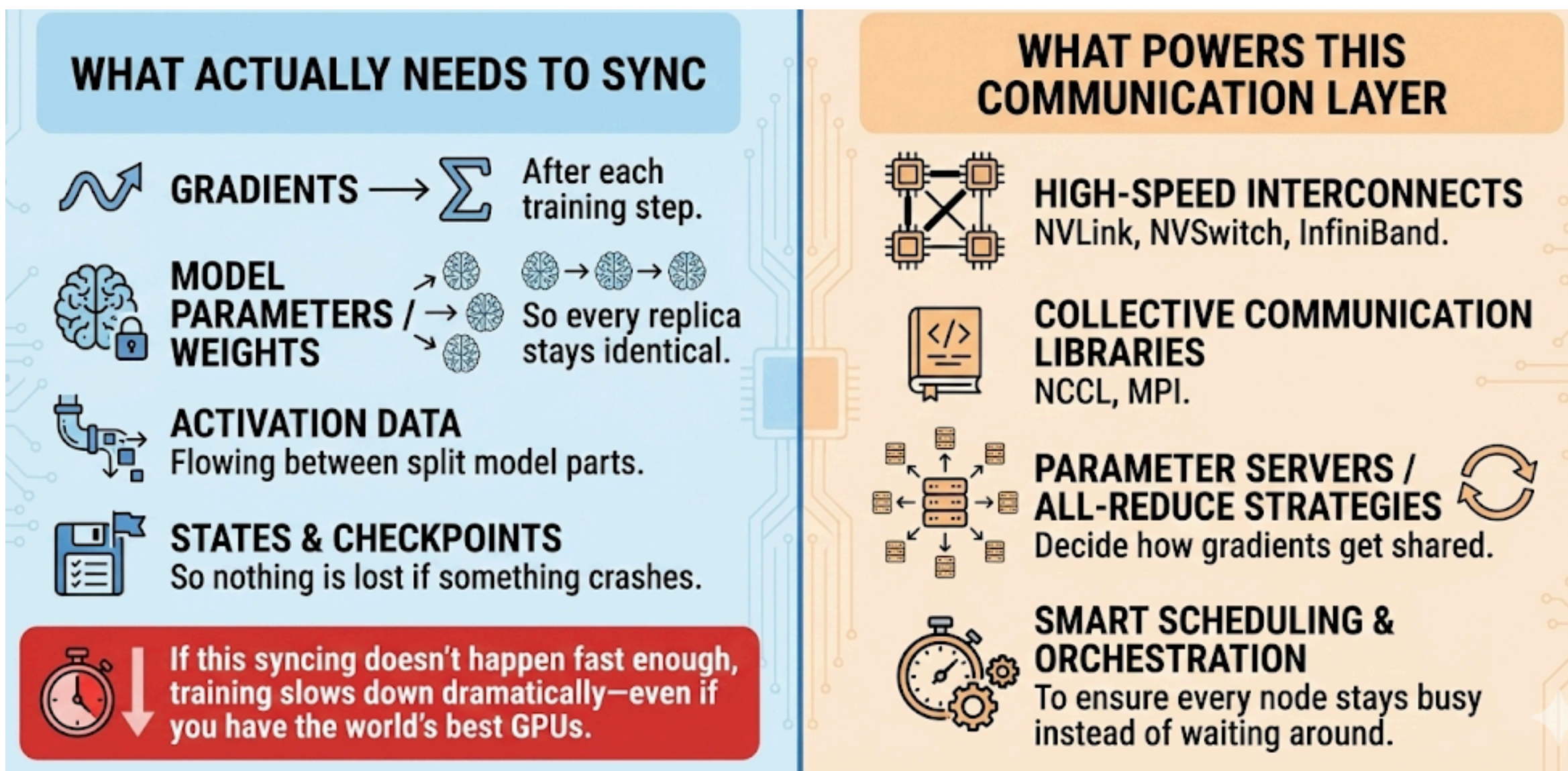
Source

What this combo really looks like in practice:

- Data Parallelism → multiple copies of the model handle different chunks of data.
- Model Parallelism → splits massive model weights across GPUs so it can even exist.
- Pipeline Parallelism → keeps GPUs busy by flowing micro-batches.

Communication & Sync

Distributed AI isn't just about more GPUs. It's about how well those GPUs talk to each other without wasting time. Communication is literally the backbone if it's slow, everything collapses into lag, inefficiency, and sadness.

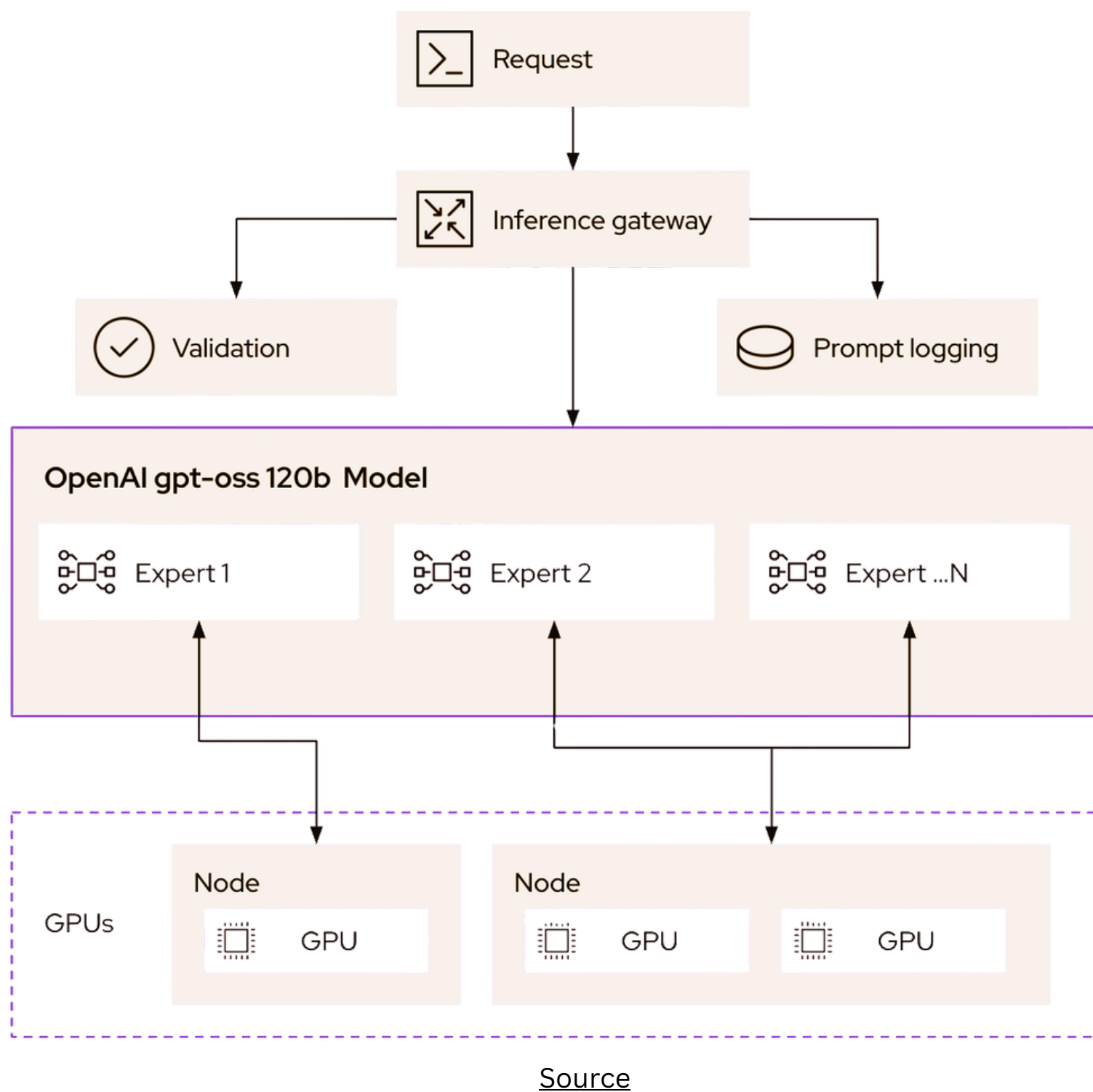


Big design challenges teams deal with:

- Latency delays between GPUs
- Bandwidth limits when gradients are huge
- Synchronization overhead when too many nodes join
- Avoiding “stragglers” slow GPUs holding everyone hostage

Distributed Inference

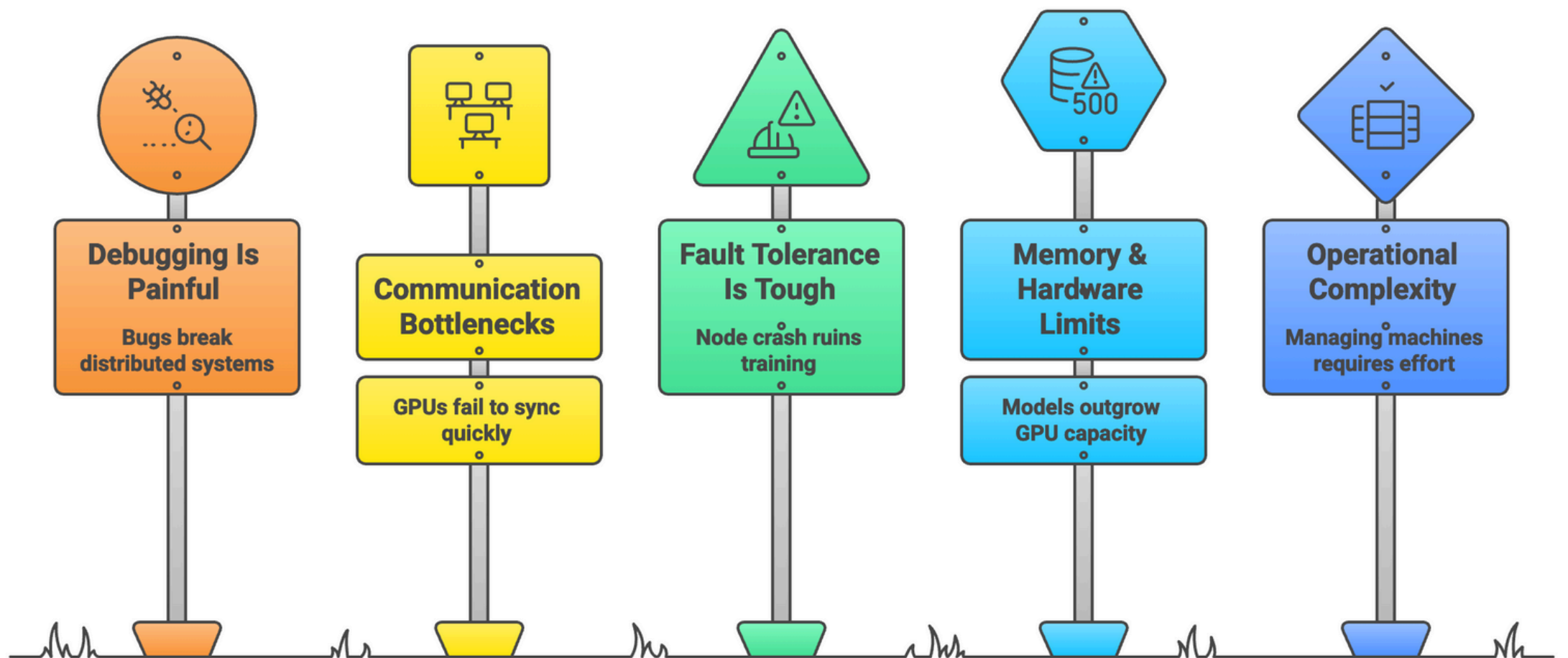
Training a massive AI model is only half the story. The real chaos begins when millions of users start hitting it at once and everyone expects instant responses. That's where distributed inference saves the day by spreading the load so the system doesn't choke.



Where we see this in the real world:

- Chatbots and LLMs handling millions of conversations
- Recommendation engines responding in milliseconds
- Real-time vision systems in autonomous driving
- Generative AI tools serving images, text, video at scale

Challenges



- **Debugging Is painful:** Bugs at distributed scale aren't simple. One slow node or sync failure can break everything.
- **Communication bottlenecks:** If GPUs can't sync fast enough, performance drops and hardware sits idle.
- **Fault tolerance is tough:** A single node crash can ruin hours of training without solid checkpointing.
- **Memory & Hardware limits:** Models grow faster than GPUs do. Engineers constantly fight VRAM limits.
- **Operational complexity:** Monitoring, scaling, managing hundreds of machines = serious engineering effort.

Stay Ahead with Our Tech Newsletter! 🚀

👉 Join 1.1k+ leaders and professionals to stay ahead in GenAI!

<https://bhavishyapandit9.substack.com/>

Join our newsletter for:

- Step-by-step guides to mastering complex topics
- Industry trends & innovations delivered straight to your inbox
- Actionable tips to enhance your skills and stay competitive
- Insights on cutting-edge AI & software development

WTF In Tech

Home

Notes

Archive

About

People with no idea about AI
saying it will take over the world:

My Neural Network:



Object Detection with Large Vision Language Models (LVLMs)

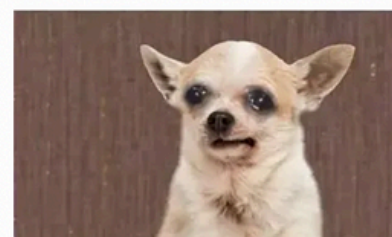
Object detection, now smarter with LVLMs

MAR 27 • BHAVISHYA PANDIT

AI Interview Playbook : Comprehensive guide to land an AI job in 2025

Brownie point: It includes 10 Key AI Interview Questions (With Answers).

MAR 22 • BHAVISHYA PANDIT



WTF In Tech

My personal Substack

💡 Whether you're a developer, researcher, or tech enthusiast, this newsletter is your shortcut to staying informed and ahead of the curve.



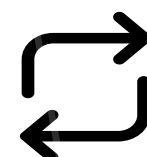
*Follow to stay updated on
Generative AI*



LIKE



COMMENT



REPOST